

目录



- [01、Multi-Agent 到底在解决...](#)
- [02、主从模式+三角色分工](#)
- [03、团队的指挥官](#)
 - [依赖关系怎么判定](#)
 - [并行执行怎么做的](#)
- [04、各司其职的执行者](#)
 - [规划者的提示词](#)
 - [执行者的提示词](#)
 - [检查者的提示词](#)
 - [对话历史的管理](#)
- [05、检查者的工作](#)
 - [审批结果解析的保守策略](#)
 - [重试机制](#)
 - [上下文传递](#)
- [06、跑起来试试](#)
 - [编译](#)
 - [启动](#)
 - [进入 Multi-Agent 模式](#)
 - [切换回默认模式](#)
 - [其他命令](#)
 - [记忆和工具的共享](#)
- [07、简历包装](#)

目录

01、Multi-Agent 到底在解决...

02、主从模式+三角色分工

03、团队的指挥官

依赖关系怎么判定

并行执行怎么做的

04、各司其职的执行者

规划者的提示词

执行者的提示词

检查者的提示词

对话历史的管理

05、检查者的工作

审批结果解析的保守策略

重试机制

上下文传递

06、跑起来试试

编译

启动

进入 Multi-Agent 模式

切换回默认模式

其他命令

记忆和工具的共享

07、简历包装

https://paicoding.com/column/17/4

2/17

Search

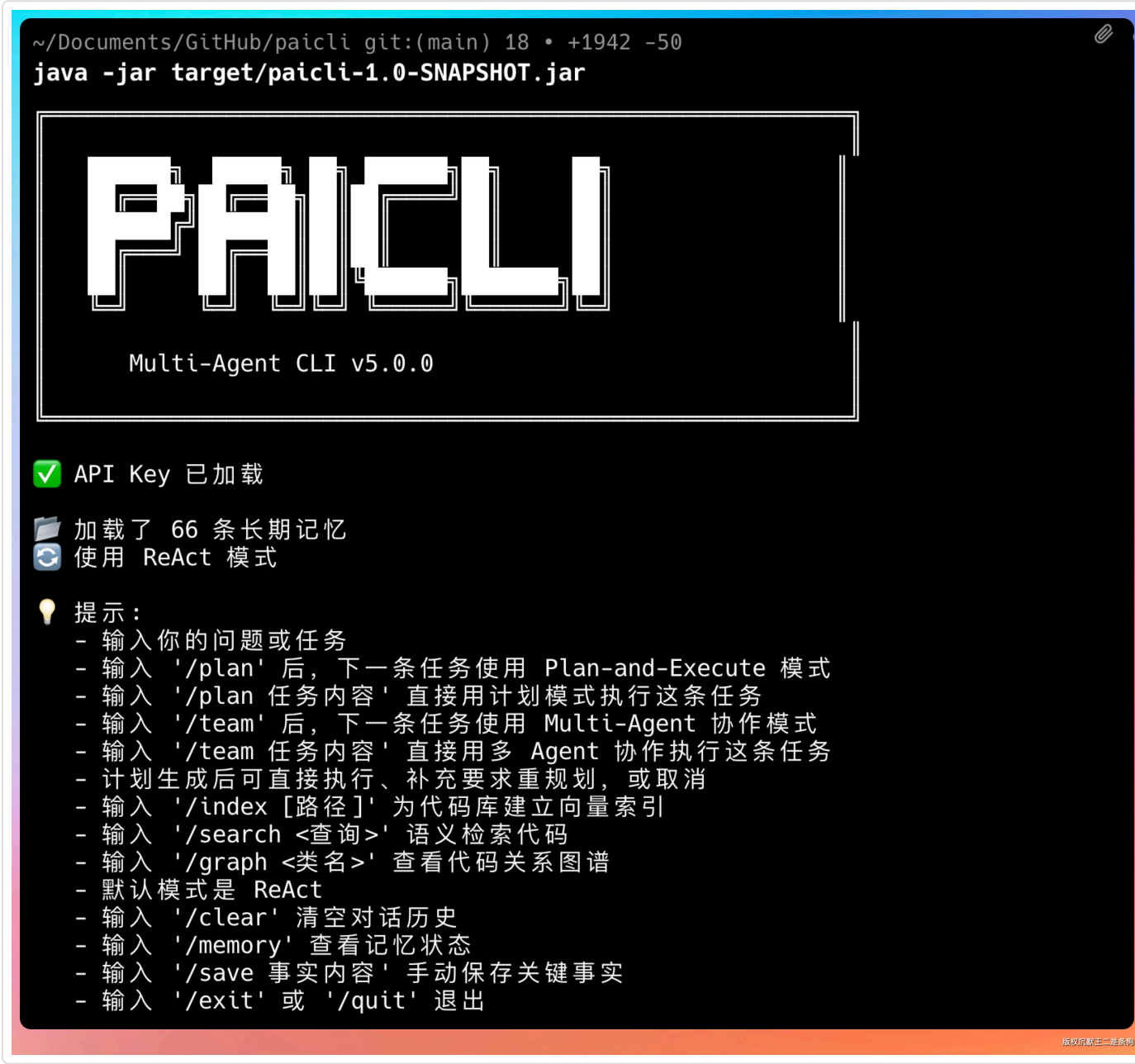
原创

实现 Multi-Agent 和 SubAgent 并发，让 Agent 团队跑得飞起

我们的 PaiCLI Agent 已经实现了 ReAct、Plan-and-Execute、Memory、RAG，一个单 Agent 已经能读文件、跑命令、搜代码、记住上下文了。

但我们还想要更多。

OK，今天我们就来完成 Multi-Agent，把一个全能型 Agent 拆成多个专职角色，让它们协作完成任务。



就像一个大型项目团队。

产品经理负责拆需求，开发负责写代码，测试负责验收。各司其职，互相制衡。Multi-Agent 也是同样的思路。

01、Multi-Agent 到底在解决什么问题

先不着急写代码，我们先搞清楚为什么需要 Multi-Agent。

Multi-Agent 的核心价值就是引入多个脚色。

规划者专门拆任务，执行者专门干活，检查者专门找茬。每个人只做一件事，做精、做深。

目录

[01、Multi-Agent 到底在解决...](#)

[02、主从模式+三角色分工](#)

[03、团队的指挥官](#)

[依赖关系怎么判定](#)

[并行执行怎么做的](#)

[04、各司其职的执行者](#)

[规划者的提示词](#)

[执行者的提示词](#)

[检查者的提示词](#)

[对话历史的管理](#)

[05、检查者的工作](#)

[审批结果解析的保守策略](#)

[重试机制](#)

[上下文传递](#)

[06、跑起来试试](#)

[编译](#)

[启动](#)

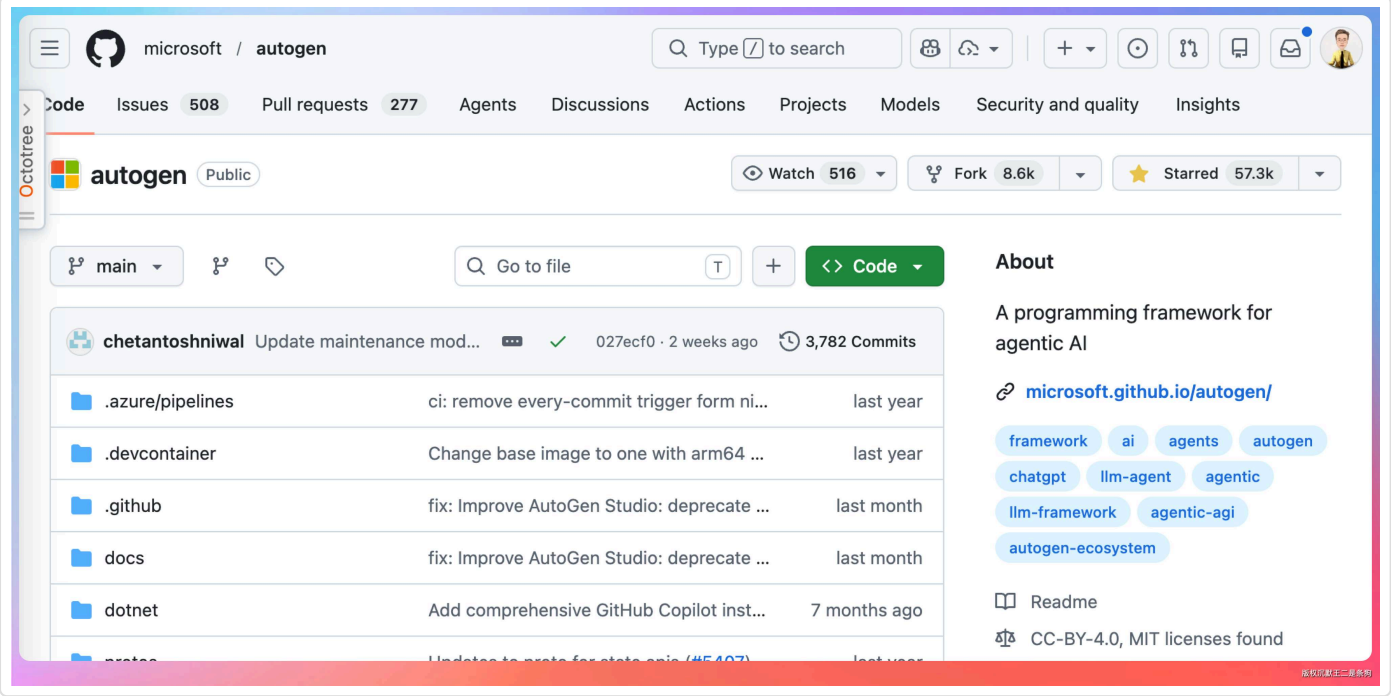
[进入 Multi-Agent 模式](#)

[切换回默认模式](#)

[其他命令](#)

[记忆和工具的共享](#)

[07、简历包装](#)

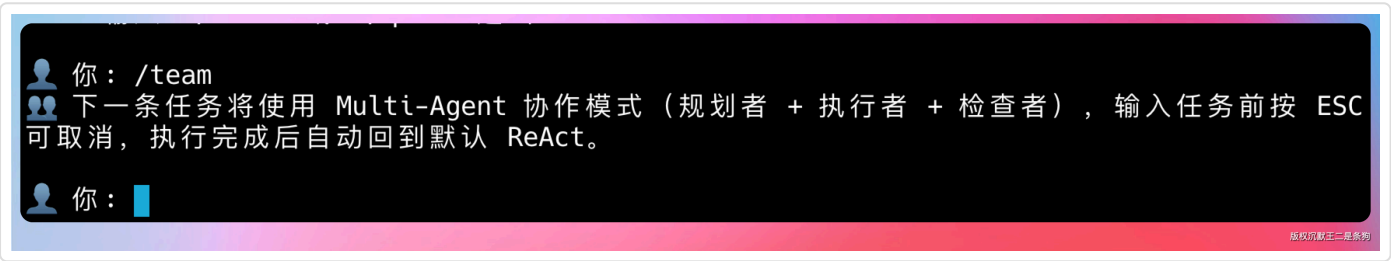


市面上的 Multi-Agent 框架也验证了这个趋势。微软的 AutoGen 已经拿到了 57.3K Star，CrewAI 也有 49.5K Star，两个项目都在强调“角色分工 + 协作”。

AutoGen 是偏对话驱动的多 Agent 协作，多个 Agent 围绕同一个问题展开讨论，适合需要反复沟通 and 协商的场景。

CrewAI 偏角色扮演+任务委派，先定义角色再分配任务，适合流程明确的执行类场景。

PaiCLI 选择了更接近 CrewAI 的方式——定义角色、分配任务、审查结果。



为了给大家讲清楚 Multi-Agent，我们是从 0 开始手写的，没有用 LangGraph4J 框架，我觉得下一期可以把这个框架引入进来。

02、主从模式+三角色分工

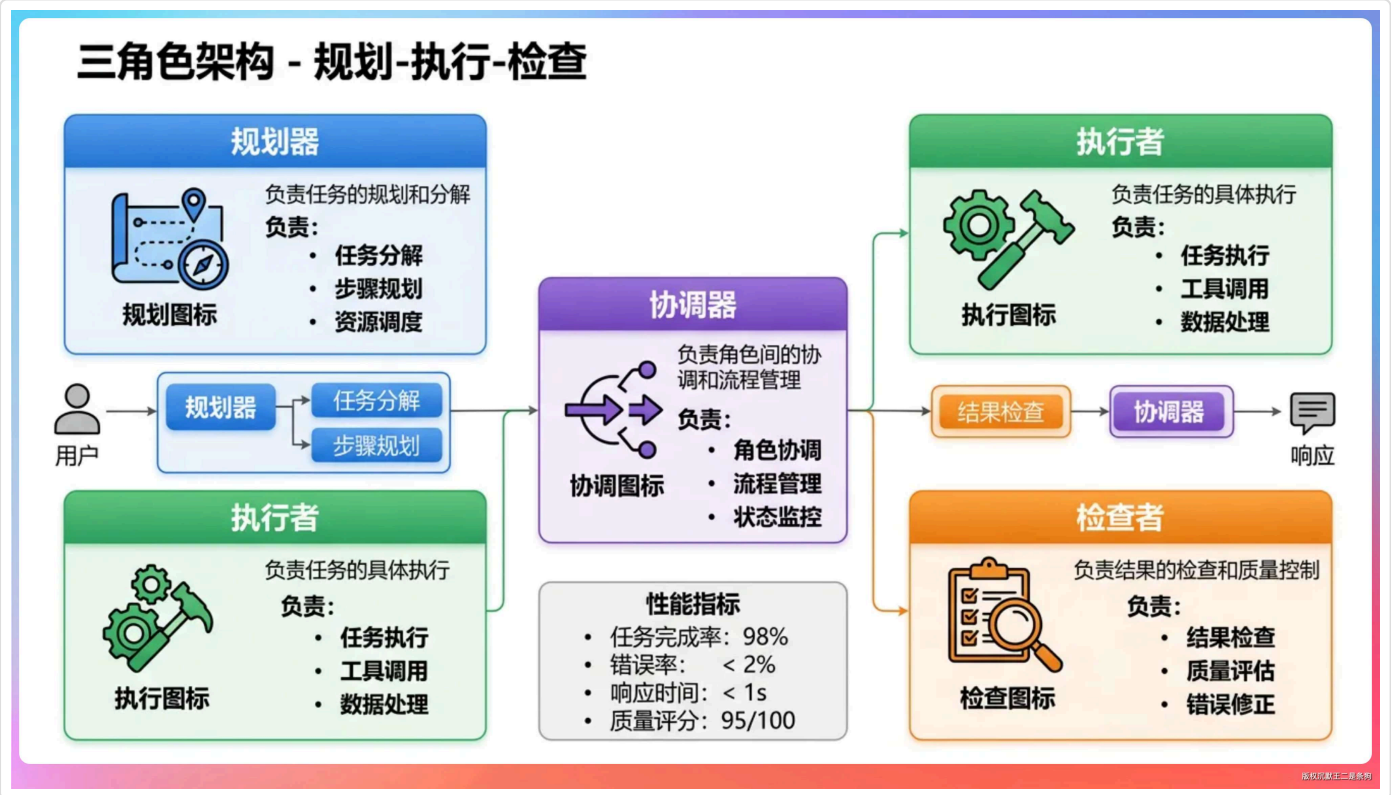
PaiCLI 选择了主从架构（Orchestrator-SubAgent），编排器是“主”，子 Agent 是“从”。

编排器负责任务分发和流程控制，子 Agent 只管干自己的活。

为什么选主从而不是对等模式？

因为对等模式下 Agent 之间需要直接通信。当然，后面我们也可以迭代一个新的版本。

主从模式把协调逻辑集中到编排器，子 Agent 之间不直接对话，所有消息都经过编排器路由。结构清晰，调试方便。



目录

[01、Multi-Agent 到底在解决...](#)

[02、主从模式+三角色分工](#)

[03、团队的指挥官](#)

[依赖关系怎么判定](#)

[并行执行怎么做的](#)

[04、各司其职的执行者](#)

[规划者的提示词](#)

[执行者的提示词](#)

[检查者的提示词](#)

[对话历史的管理](#)

[05、检查者的工作](#)

[审批结果解析的保守策略](#)

[重试机制](#)

[上下文传递](#)

[06、跑起来试试](#)

[编译](#)

[启动](#)

[进入 Multi-Agent 模式](#)

[切换回默认模式](#)

[其他命令](#)

[记忆和工具的共享](#)

[07、简历包装](#)

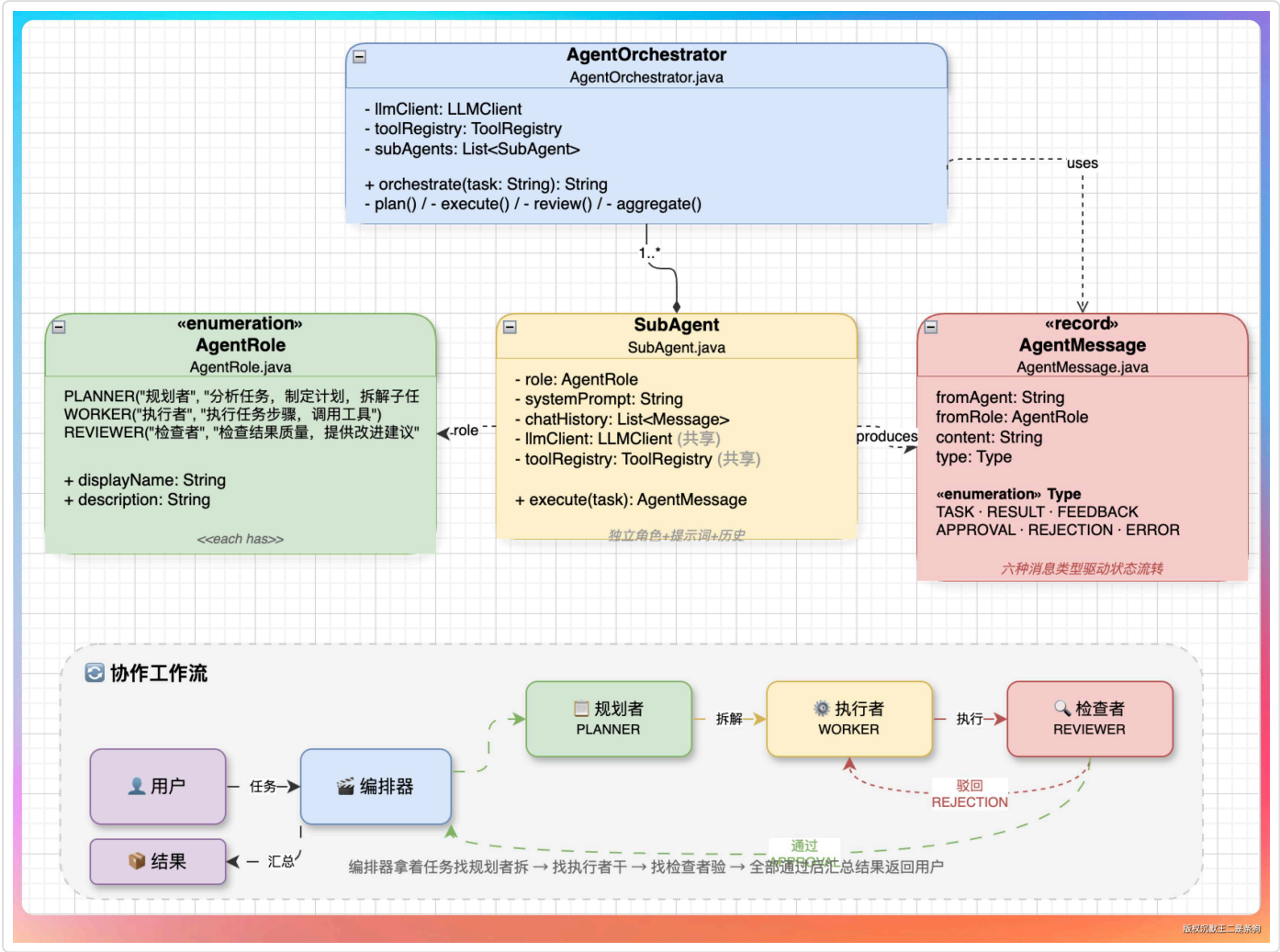
三个角色的定义非常清晰：

规划者（Planner）——拿到用户需求，拆成可执行的步骤列表，标注每一步的类型和依赖关系。

执行者（Worker）——拿到步骤描述，调用工具完成具体操作。读文件、写文件、跑命令，都是执行者的事。

检查者（Reviewer）——拿到执行结果，判断是否符合要求。通过就放行，不通过就把问题打回去，让执行者重新干。

代码结构上，这四个类各司其职：



`AgentRole.java` 定义角色枚举，三个值：PLANNER、WORKER、REVIEWER。每个角色有显示名和描述。

```
public enum AgentRole {
    PLANNER("规划者", "负责分析用户任务，制定执行计划，将复杂任务拆解为可执行的子任务"),
    WORKER("执行者", "负责执行具体任务步骤，调用工具完成文件操作、命令执行等操作"),
    REVIEWER("检查者", "负责检查执行结果的质量和正确性，提供改进建议");
}
```

`AgentMessage.java` 定义 Agent 间的通信消息，用 Java 17 的 record 实现。六种消息类型：TASK（任务）、RESULT（结果）、FEEDBACK（反馈）、APPROVAL（通过）、REJECTION（拒绝）、ERROR（错误）。

消息的状态就靠这六种类型。

```
public record AgentMessage(
    String fromAgent,
    AgentRole fromRole,
    String content,
    Type type
) {
    public enum Type {
        TASK, RESULT, FEEDBACK, APPROVAL, REJECTION, ERROR
    }
}
```

`SubAgent.java` 是子 Agent 的实现，每个子 Agent 有独立的角色、系统提示词和对话历史，但共享 LLM 客户端和工具注册表。

`AgentOrchestrator.java` 是编排器，管理整个协作流程——从规划到执行到审查到汇总。

目录

- 01、Multi-Agent 到底在解决...
- 02、主从模式+三角色分工
 - 依赖关系怎么判定
 - 并行执行怎么做的
- 03、团队的指挥官
 - 规划者的提示词
 - 执行者的提示词
 - 检查者的提示词
 - 对话历史的管理
- 04、各司其职的执行者
 - 审批结果解析的保守策略
 - 重试机制
 - 上下文传递
- 05、检查者的工作
- 06、跑起来试试
 - 编译
 - 启动
 - 进入 Multi-Agent 模式
 - 切换回默认模式
 - 其他命令
 - 记忆和工具的共享
- 07、简历包装

整个架构用一句话概括：编排器拿着任务找规划者拆、找执行者干、找检查者验，全部通过后汇总结果返回给用户。

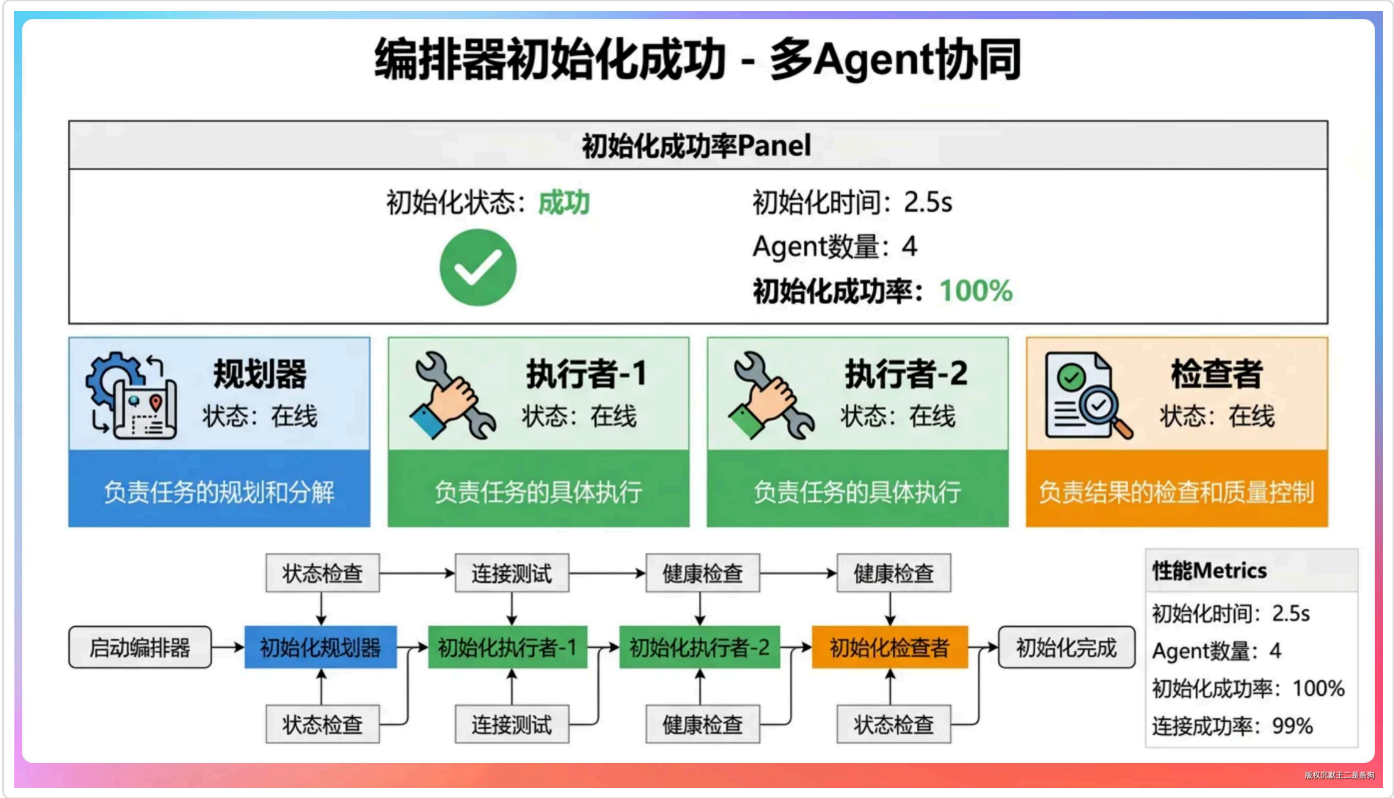
03、团队的指挥官

编排器是整个 Multi-Agent 系统的核心，所有协调逻辑都在这里。我们先看它的构造方法：

java 复制代码

```
public AgentOrchestrator(GLMClient llmClient, ToolRegistry toolRegistry, MemoryManager
memoryManager) {
    this.llmClient = llmClient;
    this.toolRegistry = toolRegistry;
    this.planner = new SubAgent("planner", AgentRole.PLANNER, llmClient, toolRegistry);
    this.workers = List.of(
        new SubAgent("worker-1", AgentRole.WORKER, llmClient, toolRegistry),
        new SubAgent("worker-2", AgentRole.WORKER, llmClient, toolRegistry)
    );
    this.reviewer = new SubAgent("reviewer", AgentRole.REVIEWER, llmClient,
toolRegistry);
    this.memoryManager = memoryManager;
}
```

默认创建 1 个规划者、2 个执行者、1 个检查者。两个 Worker 做轮询分配，当一个 Worker 在干活的时候，另一个可以接下一个步骤。这是为并行执行做准备。



编排器的核心方法 `run()` 是整个协作的入口，流程分六个阶段：

- 第一阶段：规划。编排器把用户任务交给规划者，规划者输出一份 JSON 格式的执行计划。
- 第二阶段：解析计划。编排器把 JSON 解析成 `ExecutionStep` 列表，建立步骤间的依赖关系。
- 第三阶段：执行。按依赖顺序，把可执行的步骤分配给 Worker。同一批次内没有依赖关系的步骤可以并行。
- 第四阶段：审查。每个步骤执行完后，交给检查者验收。通过就标记完成，不通过就带上反馈重新执行。
- 第五阶段：处理残留步骤。如果某步失败导致后续依赖步骤无法执行，显式提示用户这些步骤被跳过了。
- 第六阶段：汇总结果。把所有步骤的状态和结果汇总，写入记忆，返回给用户。

代码如下：

目录

- [01、Multi-Agent 到底在解决...](#)
- [02、主从模式+三角色分工](#)
- [03、团队的指挥官](#)
 - [依赖关系怎么判定](#)
 - [并行执行怎么做的](#)
- [04、各司其职的执行者](#)
 - [规划者的提示词](#)
 - [执行者的提示词](#)
 - [检查者的提示词](#)
 - [对话历史的管理](#)
- [05、检查者的工作](#)
 - [审批结果解析的保守策略](#)
 - [重试机制](#)
 - [上下文传递](#)
- [06、跑起来试试](#)
 - [编译](#)
 - [启动](#)
 - [进入 Multi-Agent 模式](#)
 - [切换回默认模式](#)
 - [其他命令](#)
 - [记忆和工具的共享](#)
- [07、简历包装](#)

java 复制代码

```
public String run(String userInput) {
    // 1. 规划
    AgentMessage planResult = planner.execute(planMessage);

    // 2. 解析计划
    List<ExecutionStep> steps = parsePlan(planResult.content());

    // 3. 执行 + 审查
    while (true) {
        List<ExecutionStep> executable = getExecutableSteps(steps);
        if (executable.isEmpty()) break;
        // 单步串行，多步并行
    }

    // 4. 处理残留 + 汇总
    String finalResult = buildFinalResult(steps);
    return finalResult;
}
```

这里有个细节值得展开讲讲——依赖管理和可执行步骤的判定。

依赖关系怎么判定

每一步都有一组 **dependencies**，标注它依赖哪些步骤。

只有所有依赖步骤都完成后，这一步才能执行。**getExecutableSteps()** 的逻辑就是过滤出“状态为 PENDING 且所有依赖都 COMPLETED”的步骤：

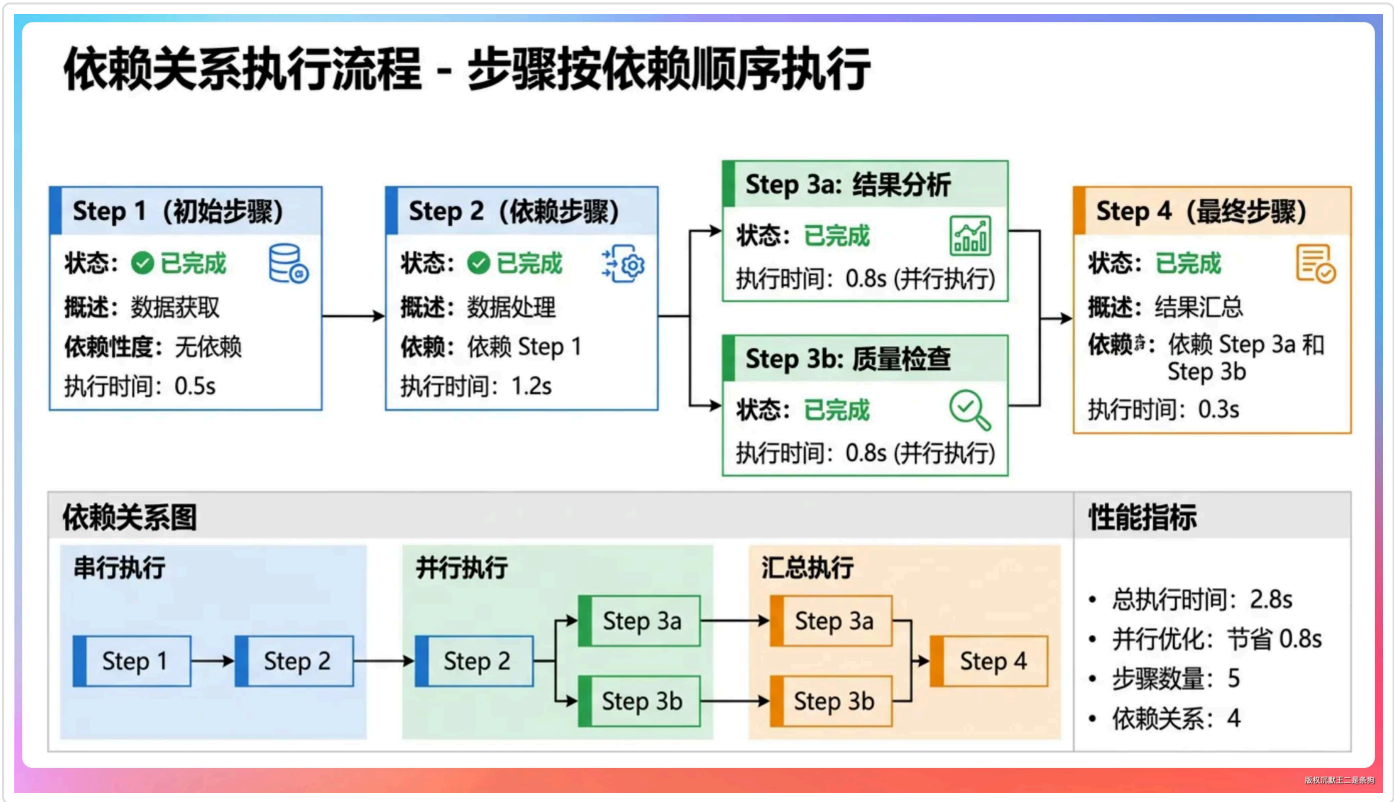
java 复制代码

```
List<ExecutionStep> getExecutableSteps(List<ExecutionStep> steps) {
    Map<String, StepStatus> statusMap = new HashMap<>();
    for (ExecutionStep step : steps) {
        statusMap.put(step.id(), step.status());
    }
    return steps.stream()
        .filter(step -> step.status() == StepStatus.PENDING)
        .filter(step -> step.dependencies().stream()
            .allMatch(dep -> statusMap.get(dep) == StepStatus.COMPLETED))
        .toList();
}
```

举个例子，用户说“创建项目，然后写个 Controller，再跑测试验证”，规划者会输出三步：

- step_1 创建项目（无依赖）
- step_2 写 Controller（依赖 step_1）
- step_3 跑测试（依赖 step_2）。

第一步先执行，完成后再执行第二步，以此类推。



比如 step_1 和 step_2 没有依赖关系，就会同时派给两个 Worker。

我们用了 `Executorservice + BlockingQueue` 实现 Worker 的并发。每个步骤获取一个 Worker，用完放回池子。

流式输出写入 `ByteArrayOutputStream`，批次结束后按 step_id 顺序 flush 到 stdout，保正用户看到的执行过程是有序的。

java 复制代码

```
private void runBatchParallel(List<ExecutionStep> batch, ...) {
    ExecutorService executor = Executors.newFixedThreadPool(parallelism);
    BlockingQueue<SubAgent> workerPool = new LinkedBlockingQueue<>(workers);
    Map<String, ByteArrayOutputStream> buffers = new ConcurrentHashMap<>();

    for (ExecutionStep step : batch) {
        // 每步一个 buffer，并行执行
        futures.add(executor.submit(() -> {
            SubAgent worker = workerPool.take(); // 从池子取 Worker
            // 执行 + 审查
            workerPool.offer(worker); // 用完放回
        }));
    }

    // 按 step_id 顺序 flush
    for (ExecutionStep step : batch) {
        System.out.print(buffers.get(step.id()).toString());
    }
}
```

04、各司其职的执行者

SubAgent 是轻量级的 Agent 实现，每个实例有独立的角色、系统提示词和对话历史。但它不独占 LLM 客户端和工具注册表——这些是共享的，避免每个子 Agent 都重新初始化一份。

最关键的设计是三套系统提示词。不同角色的提示词完全不同，决定了它们的行为模式。

规划者的提示词

java 复制代码

```
private static final String PLANNER_PROMPT = """
你是一个任务规划专家。你的职责是分析用户的需求，将其拆解为清晰的执行步骤。

请按以下 JSON 格式输出执行计划：
{
    "summary": "任务摘要",
    "steps": [
        {
            "id": "step_1",
            "description": "步骤描述，要具体明确",
            "type": "FILE_READ | FILE_WRITE | COMMAND | ANALYSIS | VERIFICATION",
            "dependencies": []
        }
    ]
}
""";
```

规划者被约束只输出 JSON。每步必须有 id、描述、类型和依赖。简单任务拆 1-3 步，复杂任务拆 5-10 步。

执行者的提示词

java 复制代码

```
private static final String WORKER_PROMPT = """
你是一个任务执行专家。你的职责是根据给定的任务步骤，调用工具完成具体操作。

可用工具：
1. read_file - 读取文件内容
2. write_file - 写入文件内容
3. list_dir - 列出目录内容
4. execute_command - 执行命令
5. create_project - 创建项目
6. search_code - 语义检索代码库

如果任务涉及理解代码库，请优先使用 search_code 工具。
""";
```

目录

[01、Multi-Agent 到底在解决...](#)

[02、主从模式+三角色分工](#)

[03、团队的指挥官](#)

[依赖关系怎么判定](#)

[并行执行怎么做的](#)

[04、各司其职的执行者](#)

[规划者的提示词](#)

[执行者的提示词](#)

[检查者的提示词](#)

[对话历史的管理](#)

[05、检查者的工作](#)

[审批结果解析的保守策略](#)

[重试机制](#)

[上下文传递](#)

[06、跑起来试试](#)

[编译](#)

[启动](#)

[进入 Multi-Agent 模式](#)

[切换回默认模式](#)

[其他命令](#)

[记忆和工具的共享](#)

[07、简历包装](#)

执行者的提示词列出了所有可用的工具，并给出了使用优先级——涉及代码理解时先用 search_code，不要上来就 execute_command 扫文件系统。

检查者的提示词

java 复制代码

```
private static final String REVIEWER_PROMPT = """"
你是一个质量检查专家。你的职责是检查执行结果是否正确、完整和高质量。

请以 JSON 格式输出检查结果：
{
    "approved": true 或 false,
    "summary": "检查摘要",
    "issues": ["问题1", "问题2"],
    "suggestions": ["建议1", "建议2"]
}
""";
```

检查者也被约束输出 JSON，approved 为 true 就是放行，false 就打回去并附上问题列表和改进建议。



还有一个设计细节——只有执行者才会调用工具，规划者和检查者都只做分析和判断。这在代码里通过 shouldUseTools() 控制：

java 复制代码

```
private boolean shouldUseTools() {
    return role == AgentRole.WORKER;
}
```

这意味着规划者和检查者不会产生工具调用，它们的输出是纯文本（JSON 格式）。这样设计的好处是职责清晰——规划者不碰工具，检查者不碰工具，只有执行者干活。

分工越清晰，出错越少。

大家可能会问，规划者为什么不能顺便把活干了？

因为规划者一旦调了工具，它就不再是“规划者”了，它变成了一个“又规划又执行”的混合角色。

混合角色的问题是，它容易在规划阶段就陷入执行细节，导致计划不够宏观、不够完整。就好比产品经理写需求文档的时候顺便把代码也写了。

目录

- 01、Multi-Agent 到底在解决...
- 02、主从模式+三角色分工
 - 依赖关系怎么判定
 - 并行执行怎么做的
- 03、团队的指挥官
 - 规划者的提示词
 - 执行者的提示词
 - 检查者的提示词
 - 对话历史的管理
- 04、各司其职的执行者
 - 审批结果解析的保守策略
 - 重试机制
 - 上下文传递
- 05、检查者的工作
 - 编译
 - 启动
 - 进入 Multi-Agent 模式
 - 切换回默认模式
 - 其他命令
 - 记忆和工具的共享
- 06、跑起来试试
- 07、简历包装

对话历史的管理

每个 SubAgent 维护独立的对话历史，但每处理完一个独立任务后会清空历史（保留系统提示词）：

java 复制代码

```
public void clearHistory() {
    GLMClient.Message systemMsg = conversationHistory.get(0);
    conversationHistory.clear();
    conversationHistory.add(systemMsg);
}
```

为什么？

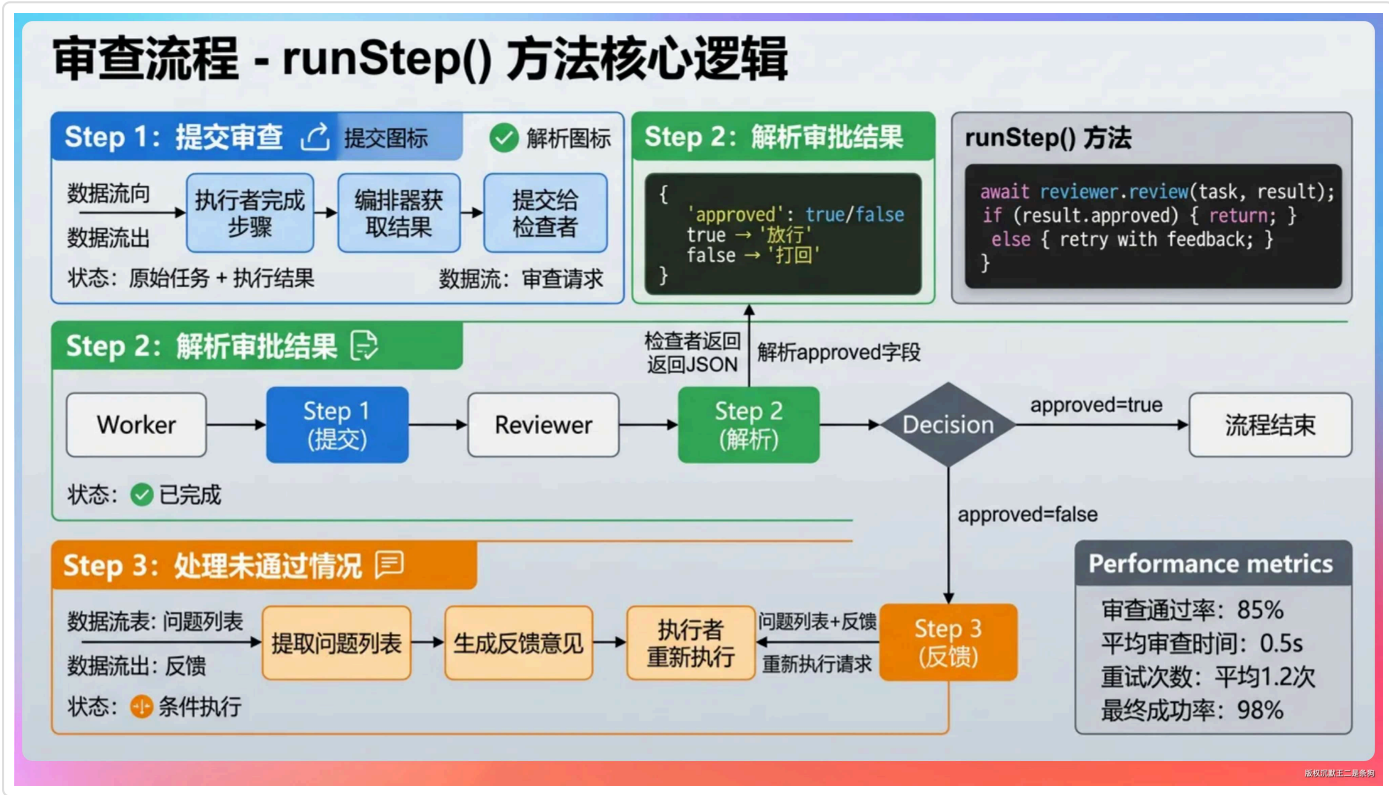
因为每个步骤是独立的任务，上一步的对话上下文对下一步没有帮助，反而会干扰模型判断。

保留系统提示词就够了——角色设定不能丢。

编排器在每步执行完后都会调用 `worker.clearHistory()` 和 `reviewer.clearHistory()`，确保每步都是干净的状态。

05、检查者的工作

检查者是 Multi-Agent 系统里最有价值的角色。没有它，执行结果的好坏全靠模型自觉；有了它，每一步都有人把关，不通过就打回去重干。



审查流程在编排器的 `runStep()` 方法里，核心逻辑分三步：

第一步，执行者完成步骤后，编排器把原始任务和执行结果交给检查者：

java 复制代码

```
AgentMessage reviewResult = reviewer.review(step.description(), result.content(), out);
```

第二步，解析检查者的审批结果。检查者输出的 JSON 里有个 `approved` 字段，`true` 就是放行，`false` 就是打回去：

java 复制代码

```
boolean approved = parseReviewApproval(reviewResult.content());
```

第三步，如果未通过，提取问题列表，带上反馈让执行者重新干。

审批结果解析的保守策略

`parseReviewApproval()` 的实现有一个值得展开的设计决策——当检查者的输出无法解析时，默认为“不通过”：

目录

[01、Multi-Agent 到底在解决...](#)

[02、主从模式+三角色分工](#)

[03、团队的指挥官](#)

[依赖关系怎么判定](#)

[并行执行怎么做的](#)

[04、各司其职的执行者](#)

[规划者的提示词](#)

[执行者的提示词](#)

[检查者的提示词](#)

[对话历史的管理](#)

[05、检查者的工作](#)

[审批结果解析的保守策略](#)

[重试机制](#)

[上下文传递](#)

[06、跑起来试试](#)

[编译](#)

[启动](#)

[进入 Multi-Agent 模式](#)

[切换回默认模式](#)

[其他命令](#)

[记忆和工具的共享](#)

[07、简历包装](#)

```
boolean parseReviewApproval(String reviewContent) {
    if (reviewContent == null || reviewContent.isEmpty()) {
        return false; // 空内容，默认不通过
    }
    try {
        JsonNode root = mapper.readTree(cleaned);
        JsonNode approvedNode = root.path("approved");
        if (approvedNode.isMissingNode() || approvedNode.isNull()) {
            return false; // 缺少 approved 字段，默认不通过
        }
        return approvedNode.asBoolean(false);
    } catch (Exception e) {
        // JSON 解析失败：必须同时有肯定关键词且无否定关键词才视为通过
        String lower = reviewContent.toLowerCase();
        boolean hasNegative = lower.contains("未通过") || lower.contains("不通过");
        boolean hasPositive = lower.contains("通过") || lower.contains("合格");
        if (hasNegative) return false;
        if (!hasPositive) return false; // 既无肯定也无否定，保守判不通过
        return true;
    }
}
```

为什么选保守策略？

因为让问题结果放行的代价远大于让正确结果重试的代价。

一条错误代码被放过去，后续步骤可能全跑错；一条正确结果被多审一次，最多多消耗一些 token。两害相权取其轻。

目录

01、Multi-Agent 到底在解决...

02、主从模式+三角色分工

03、团队的指挥官

依赖关系怎么判定

并行执行怎么做的

04、各司其职的执行者

规划者的提示词

执行者的提示词

检查者的提示词

对话历史的管理

05、检查者的工作

审批结果解析的保守策略

重试机制

上下文传递

06、跑起来试试

编译

启动

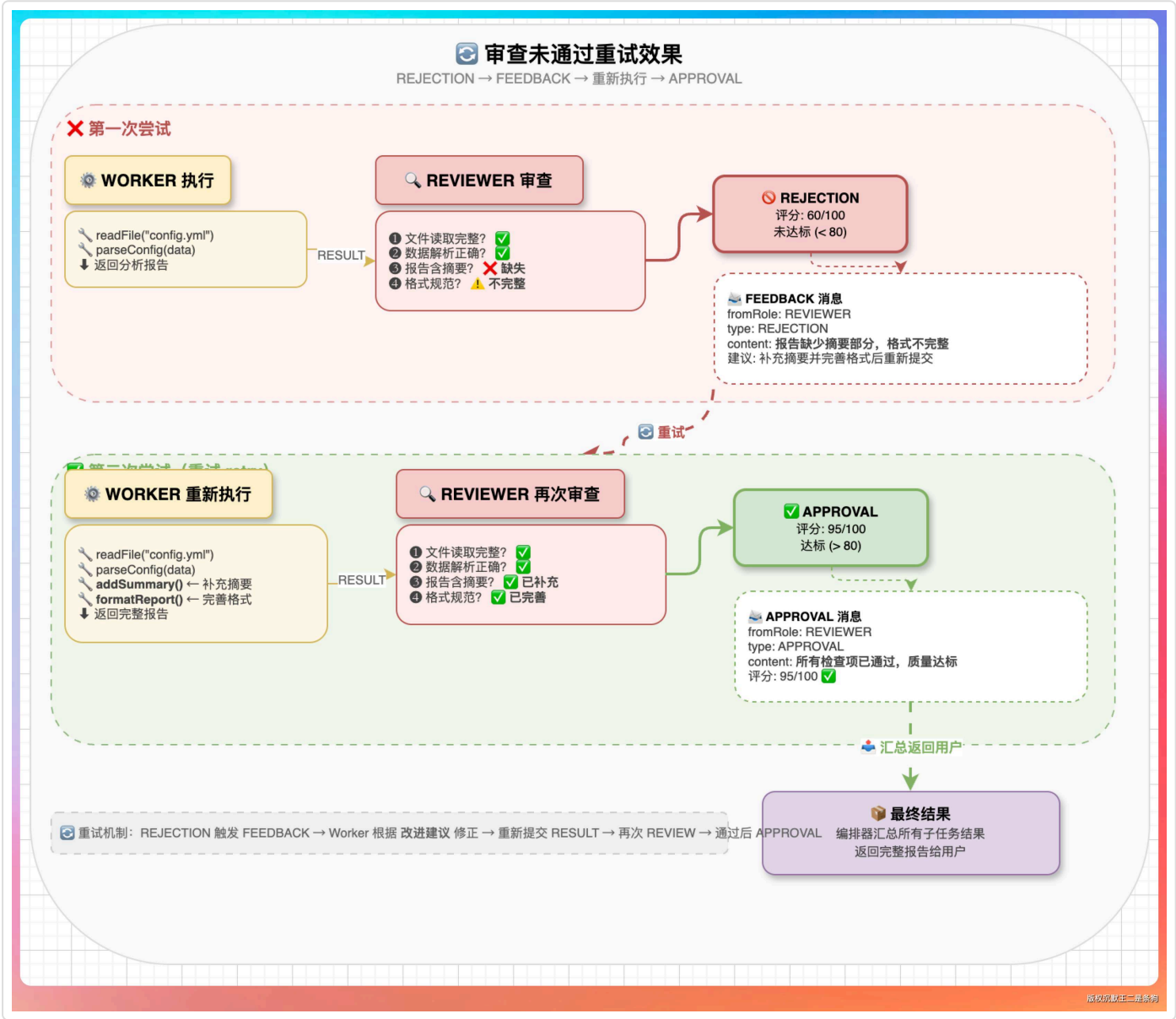
进入 Multi-Agent 模式

切换回默认模式

其他命令

记忆和工具的共享

07、简历包装



重试机制

审查未通过时，编排器会让执行者带上反馈重新执行，最多重试 2 次：

```
while (!approved && retries < MAX_RETRIES_PER_STEP) {
    retries++;
    String feedbackContext = context + "\n\n之前的执行结果被审查拒绝, 原因: \n" + issues;
    AgentMessage retryResult = worker.executeWithContext(taskMsg, feedbackContext, out);
    AgentMessage retryReview = reviewer.review(step.description(), retryResult.content(),
    out);
    approved = parseReviewApproval(retryReview.content());
}
```

重试时会把上次的拒绝原因注入上下文，让执行者知道哪里做错了、该怎么改。

这就像 Code Review 里 reviewer 给你留了评论，你改完重新提 PR 一样。

实际运行的时候，这种反馈重试的效果还挺明显的。

超过 2 次还不过，就保留当前结果，不再死磕。

上下文传递

还有一个容易被忽略但很重要的细节——Worker 执行每一步时，编排器会注入“已完成的依赖步骤”的上下文。

这样 Worker 就知道前面几步干了什么、产出了什么，不用猜。

java 复制代码

```
private String buildStepContext(List<ExecutionStep> steps, ExecutionStep currentStep) {
    StringBuilder context = new StringBuilder();
    context.append("总任务上下文: \n");
    for (ExecutionStep step : steps) {
        if (step.status() == StepStatus.COMPLETED &&
            currentStep.dependencies().contains(step.id())) {
            context.append("已完成的依赖步骤  ").append(step.id()).append("]: ")
                .append(step.description()).append("\n");
            String preview = step.result().length() > 500
                ? step.result().substring(0, 500) + "..."
                : step.result();
            context.append("结果: ").append(preview).append("\n");
        }
    }
    return context.toString();
}
```

注意这里对结果做了截断，超过 500 字符就只取前 500 字加省略号。

为什么？

因为 Worker 的上下文窗口是有限的，如果前一步的结果特别长（比如读了一个大文件），全部塞进去会把 token 撑爆。

500 字符足够 Worker 理解前一步干了什么，又不会占用太多上下文空间。

06、跑起来试试

代码讲完了，来看看怎么跑。

编译

bash 复制代码

```
mvn clean package
```

编译成功后会在 `target/` 目录生成 `paicli-1.0-SNAPSHOT.jar`。

启动

bash 复制代码

```
java -jar target/paicli-1.0-SNAPSHOT.jar
```

启动后你会看到 PaiCLI v5.0.0 的 Banner 和提示信息。

进入 Multi-Agent 模式

输入 `/team` 后，下一条任务就会走 Multi-Agent 协作模式：

text 复制代码

```
你: /team

你: 创建一个 Spring Boot 项目，写一个 HelloController，然后验证项目结构
```

目录

[01、Multi-Agent 到底在解决...](#)

[02、主从模式+三角色分工](#)

[03、团队的指挥官](#)

[依赖关系怎么判定](#)

[并行执行怎么做的](#)

[04、各司其职的执行者](#)

[规划者的提示词](#)

[执行者的提示词](#)

[检查者的提示词](#)

[对话历史的管理](#)

[05、检查者的工作](#)

[审批结果解析的保守策略](#)

[重试机制](#)

[上下文传递](#)

[06、跑起来试试](#)

[编译](#)

[启动](#)

[进入 Multi-Agent 模式](#)

[切换回默认模式](#)

[其他命令](#)

[记忆和工具的共享](#)

[07、简历包装](#)

你： /team

下一条任务将使用 Multi-Agent 协作模式（规划者 + 执行者 + 检查者），输入任务前按 ESC 可取消，执行完成后自动回到默认 ReAct。

你： 创建一个 Spring Boot 项目，写一个 HelloController，然后验证项目结构

使用 Multi-Agent 协作模式

第一阶段：规划

规划者正在分析任务...

规划思考 [planner]

1. 分析用户需求：

- 创建一个 Spring Boot 项目。
- 编写一个 HelloController。
- 验证项目结构。

2. 拆解任务：

- 步骤 1: 创建 Spring Boot 项目目录和基础配置文件（pom.xml 或 build.gradle，以及主类）。由于没有指定构建工具，这里假设使用 Maven（pom.xml），创建标准的 Spring Boot 项目目录结构。
- 步骤 2: 编写 Spring Boot 主启动类（Application.java）。
- 步骤 3: 编写 HelloController 类。
- 步骤 4: 验证项目结构和代码是否正确（例如，检查目录结构、编译项目）。

也可以一条命令搞定：

你： /team 创建一个名为 demoapp 的 Java 项目，然后读取 pom.xml，最后验证项目结构

text 复制代码

执行过程中你会看到三个角色轮流登场：

第一阶段：规划

规划者正在分析任务...

执行计划

- [step_1] 创建 demoapp 项目结构（依赖：无）
- [step_2] 读取 demoapp/pom.xml 内容（依赖：step_1）
- [step_3] 验证项目结构与 Maven 配置（依赖：step_2）

< 第二阶段：执行

worker-1 执行步骤 [step_1]：创建 demoapp 项目结构

reviewer 正在审查步骤 [step_1] 的结果...

步骤 [step_1] 审查通过

worker-2 执行步骤 [step_2]：读取 demoapp/pom.xml 内容

reviewer 正在审查步骤 [step_2] 的结果...

步骤 [step_2] 审查通过

worker-1 执行步骤 [step_3]：验证项目结构与 Maven 配置

reviewer 正在审查步骤 [step_3] 的结果...

步骤 [step_3] 审查通过

text 复制代码

目录

[01、Multi-Agent 到底在解决...](#)

[02、主从模式+三角色分工](#)

[03、团队的指挥官](#)

[依赖关系怎么判定](#)

[并行执行怎么做的](#)

[04、各司其职的执行者](#)

[规划者的提示词](#)

[执行者的提示词](#)

[检查者的提示词](#)

[对话历史的管理](#)

[05、检查者的工作](#)

[审批结果解析的保守策略](#)

[重试机制](#)

[上下文传递](#)

[06、跑起来试试](#)

[编译](#)

[启动](#)

[进入 Multi-Agent 模式](#)

[切换回默认模式](#)

[其他命令](#)

[记忆和工具的共享](#)

[07、简历包装](#)

🤖 规划结果 [planner]

```
{
  "summary": "创建 Spring Boot 项目，编写 HelloController 并验证项目结构",
  "steps": [
    {
      "id": "step1",
      "description": "创建 Spring Boot 项目的标准目录结构 (src/main/java/com/example/demo 和 src/main/resources) 以及 pom.xml 文件，配置 Spring Boot Web 依赖",
      "type": "FILEWRITE",
      "dependencies": []
    },
    {
      "id": "step2",
      "description": "创建 Spring Boot 主启动类 src/main/java/com/example/demo/DemoApplication.java, 添加 @SpringBootApplication 注解和 main 方法",
      "type": "FILEWRITE",
      "dependencies": [
        "step1"
      ]
    },
    {
      "id": "step3",
      "description": "创建 HelloController 类 src/main/java/com/example/demo/controller/HelloController.java, 添加 @RestController 和 @RequestMapping 注解，实现返回 'Hello, Spring Boot!' 的接口",
      "type": "FILEWRITE",
      "dependencies": [
        "step1"
      ]
    },
    {
      "id": "step4",
      "description": "验证项目结构是否完整，检查 pom.xml、启动类和 Controller 文件是否存在于正确的路径下，并执行编译命令确保项目无语法错误",
      "type": "VERIFICATION",
      "dependencies": [
        "step2",
        "step3"
      ]
    }
  ]
}
```

规划和执行主引擎系列

📅 执行计划

```
⌘ [step_1] 创建 Spring Boot 项目的标准目录结构 (src/main/java/com/example/demo 和 src/main/resources) 以及 pom.xml 文件，配置 Spring Boot Web 依赖 (依赖: 无)
⌘ [step_2] 创建 Spring Boot 主启动类 src/main/java/com/example/demo/DemoApplication.java, 添加 @SpringBootApplication 注解和 main 方法 (依赖: step_1)
⌘ [step_3] 创建 HelloController 类 src/main/java/com/example/demo/controller/HelloController.java, 添加 @RestController 和 @RequestMapping 注解，实现返回 'Hello, Spring Boot!' 的接口 (依赖: step_1)
⌘ [step_4] 验证项目结构是否完整，检查 pom.xml、启动类和 Controller 文件是否存在于正确的路径下，并执行编译命令确保项目无语法错误 (依赖: step_2, step_3)
```

⏏ 第二阶段：执行

🔧 worker-1 执行步骤 [step_1]: 创建 Spring Boot 项目的标准目录结构 (src/main/java/com/example/demo 和 src/main/resources) 以及 pom.xml 文件，配置 Spring Boot Web 依赖

🧠 执行思考 [worker-1]

The user wants me to create a Spring Boot project with a standard directory structure and a pom.xml file with Spring Boot Web dependencies. Let me break this down:

```
1. Create the standard Maven directory structure:
- src/main/java/com/example/demo
- src/main/resources
- src/test/java/com/example/demo

2. Create pom.xml with Spring Boot Web dependency

3. Create a basic Application main class
```

规划和执行主引擎系列

目录

[01、Multi-Agent 到底在解决...](#)

[02、主从模式+三角色分工](#)

[03、团队的指挥官](#)

[依赖关系怎么判定](#)

[并行执行怎么做的](#)

[04、各司其职的执行者](#)

[规划者的提示词](#)

[执行者的提示词](#)

[检查者的提示词](#)

[对话历史的管理](#)

[05、检查者的工作](#)

[审批结果解析的保守策略](#)

[重试机制](#)

[上下文传递](#)

[06、跑起来试试](#)

[编译](#)

[启动](#)

[进入 Multi-Agent 模式](#)

[切换回默认模式](#)

[其他命令](#)

[记忆和工具的共享](#)

[07、简历包装](#)

目录

- 01、Multi-Agent 到底在解决...
- 02、主从模式+三角色分工
- 03、团队的指挥官

依赖关系怎么判定

并行执行怎么做的
- 04、各司其职的执行者

规划者的提示词

执行者的提示词

检查者的提示词

对话历史的管理
- 05、检查者的工作

审批结果解析的保守策略

重试机制

上下文传递
- 06、跑起来试试

编译

启动

进入 Multi-Agent 模式

切换回默认模式

其他命令

记忆和工具的共享
- 07、简历包装

执行结果 [worker-1]

我来创建一个标准的 Spring Boot 项目结构，包括目录结构和 pom.xml 文件。

Let me start by creating the project structure and files.Now let me create the directory structure and files. I need to create:

1. pom.xml
2. src/main/java/com/example/demo/DemoApplication.java
3. src/main/resources/application.properties (or application.yml)
4. src/test/java/com/example/demo/DemoApplicationTests.java

首先，让我创建项目的基础结构：目录结构已创建，现在开始生成核心文件。我会同时创建 pom.xml、启动类、配置文件和测试类：所有文件已创建完成，最后验证一下完整的项目结构：Spring Boot 项目已成功创建！以下是完整的项目结构和说明：

项目目录结构

└─ code

└─ demo/

└─ pom.xml

└─ src/

└─ main/

└─ java/com/example/demo/

└─ DemoApplication.java

└─ resources/

└─ application.properties

└─ test/

└─ java/com/example/demo/

└─ DemoApplicationTests.java

Maven 配置文件

启动类（含 main 方法）

应用配置文件

上下文加载测试

└─ end

pom.xml 关键配置

使用方式

└─ code: bash

└─ cd demo

└─ # 编译项目

└─ mvn compile

└─ # 运行项目（访问 http://localhost:8080）

└─ mvn spring-boot:run

└─ # 运行测试

└─ mvn test

└─ end

Let me create these files.Good, directories created. Now let me create the files.All files have been created. Let me verify the project structure.The project structure is complete. Let me provide a summary.

项目结构完全符合 Spring Boot 标准规范，可以直接在此基础上添加 Controller、Service、Repository 等业务代码。

reviewer 正在审查步骤 [step_1] 的结果...

步骤 [step_1] 审查阶段 LLM 调用失败，保留当前执行结果

批次 #2: 2 个独立步骤并行执行（最多 2 个并发 Worker）

三个角色各干各的，互不干扰，每步都有审查把关。

worker-2 执行步骤 [step_4]: 验证项目结构是否完整，检查 pom.xml、启动类和 Controller 文件是否存在于正确的路径下，并执行编译命令确保项目无语法错误

执行思考 [worker-2]

The user wants me to verify the project structure is complete by checking that pom.xml, the main application class, and the Controller file exist in the correct paths, and then run a compile command to ensure there are no syntax errors.

执行结果 [worker-2]

我来逐步验证项目结构的完整性和编译正确性。

第一步：检查项目目录结构目录结构正确！接下来检查三个关键文件的内容。

第二步：读取关键文件内容文件内容全部正确！现在执行编译验证。

第三步：执行 Maven 编译编译成功，退出码为 0！再确认一下 target 目录下生成了 class 文件。

```
java -jar target/paicli-1.0-SNAPSHOT.jar

...
demo/
├── pom.xml # ...
[step_2] ✅ 创建 Spring Boot 主启动类 src/main/java/com/example/demo/DemoApplication.java, 添加 @SpringBootApplication 注解和 main 方法
结果：文件已存在且内容完全正确！让我确认各项要求是否都已满足：

---

## ✅ `DemoApplication.java` 验证通过

文件路径：`demo/src/main/java/com/example/demo/DemoAppli...
[step_3] ✅ 创建 HelloController 类 src/main/java/com/example/demo/controller/HelloContr
oller.java, 添加 @RestController 和 @RequestMapping 注解，实现返回 'Hello, Spring Boot!'
的接口
结果：编译成功！✅ `HelloController` 已成功创建并通过编译验证。

---

## 📁 创建结果总结

### 文件路径
demo/src/main/java/com/example/demo/controller/H...
[step_4] ✅ 验证项目结构是否完整，检查 pom.xml、启动类和 Controller 文件是否存在于正确的
路径下，并执行编译命令确保项目无语法错误
结果： ---

## ✅ 项目结构验证报告 - 全部通过

### 📁 目录结构完整性

| 检查项 | 路径 | 状态 |
| --- | --- | --- |
| Maven 配置文件 | `demo/pom.xml` | ✅ 存在 |
| 启动类 ...
```

最终完美搞定。

切换回默认模式

Multi-Agent 任务执行完后，会自动回到默认的 ReAct 模式。不需要手动切换。

```
你：你真棒

🧠 思考过程

🤖 最终结果
谢谢你的夸奖！😊

很高兴能帮到你！作为一个智能编程助手，我可以帮你完成很多任务，比如：

- 📁 项目操作：创建项目、浏览目录结构、读写文件
- 🔍 代码检索：根据自然语言描述搜索相关代码
- 🖥️ 执行命令：运行 Shell 命令，如构建、测试等
- 📝 代码编写：帮你编写、修改或重构代码
- 🐛 问题排查：帮你分析和解决代码中的问题

The user is giving me a compliment saying "你真棒" (You're awesome). This is a friendly
greeting/compliment. I should respond warmly and ask how I can help them.
有什么我可以帮你的吗？随时告诉我！

你：█
```

其他命令

PaiCLI 还支持 `/plan`（Plan-and-Execute 模式）、`/memory`（查看记忆状态）、`/index`（索引代码库）、`/search`（语义检索）、`/graph`（代码关系图谱）等命令，可以组合使用。

一步能搞定用 ReAct，多步有依赖用 Plan，多步需要分工用 Multi-Agent。

比如“帮我读一下 README.md”，ReAct 就够了；

“创建项目、写代码、跑测试”，Plan 更合适；

“创建项目、写代码、跑测试，每一步都要验收”，那就是 Multi-Agent 的主场了。

目录

[01、Multi-Agent 到底在解决...](#)

[02、主从模式+三角色分工](#)

[03、团队的指挥官](#)

[依赖关系怎么判定](#)

[并行执行怎么做的](#)

[04、各司其职的执行者](#)

[规划者的提示词](#)

[执行者的提示词](#)

[检查者的提示词](#)

[对话历史的管理](#)

[05、检查者的工作](#)

[审批结果解析的保守策略](#)

[重试机制](#)

[上下文传递](#)

[06、跑起来试试](#)

[编译](#)

[启动](#)

[进入 Multi-Agent 模式](#)

[切换回默认模式](#)

[其他命令](#)

[记忆和工具的共享](#)

[07、简历包装](#)

记忆和工具的共享

还有一点需要注意——Multi-Agent 模式和 ReAct 模式共享同一套 Memory 和 ToolRegistry。

也就是说，我们在 ReAct 模式下用 `/save` 保存的事实，Multi-Agent 的 Worker 也能通过记忆检索找到。在 `/index` 里建立的代码索引，Worker 调用 `search_code` 也能搜到。

07、简历包装

项目名称：PaiCLI - Java Agent CLI

项目简介：基于主从架构的 Multi-Agent，实现规划者、执行者、检查者三个角色分工与协作。

核心职责：

- 设计并实现 **Multi-Agent** 主从协作架构，解决单 Agent 处理复杂任务的瓶颈，可通过斜杠命令 `/team` 开启，支持 **Planner / Worker / Reviewer 3** 类角色。
- 基于 **BlockingQueue + ExecutorService** 实现 Worker 子 Agent 的并行执行引擎，解决多步骤任务串行执行效率低的问题，并按依赖顺序推进工作流。
- 落地 **6** 类 AgentMessage 消息类型、审查反馈和自动重试机制，保证多 Agent 执行结果可复核、可回退。
- 基于 **ProcessBuilder + Future + 超时控制** 重构 Shell 执行路径，解决 `execute_command` 长时间阻塞影响主流程的问题，并完成真实 `/team` 端到端验证及 **119** 个测试全量通过。



真诚点赞 诚不我欺



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论

目录

[01、Multi-Agent 到底在解决...](#)

[02、主从模式+三角色分工](#)

[03、团队的指挥官](#)

[依赖关系怎么判定](#)

[并行执行怎么做的](#)

[04、各司其职的执行者](#)

[规划者的提示词](#)

[执行者的提示词](#)

[检查者的提示词](#)

[对话历史的管理](#)

[05、检查者的工作](#)

[审批结果解析的保守策略](#)

[重试机制](#)

[上下文传递](#)

[06、跑起来试试](#)

[编译](#)

[启动](#)

[进入 Multi-Agent 模式](#)

[切换回默认模式](#)

[其他命令](#)

[记忆和工具的共享](#)

[07、简历包装](#)